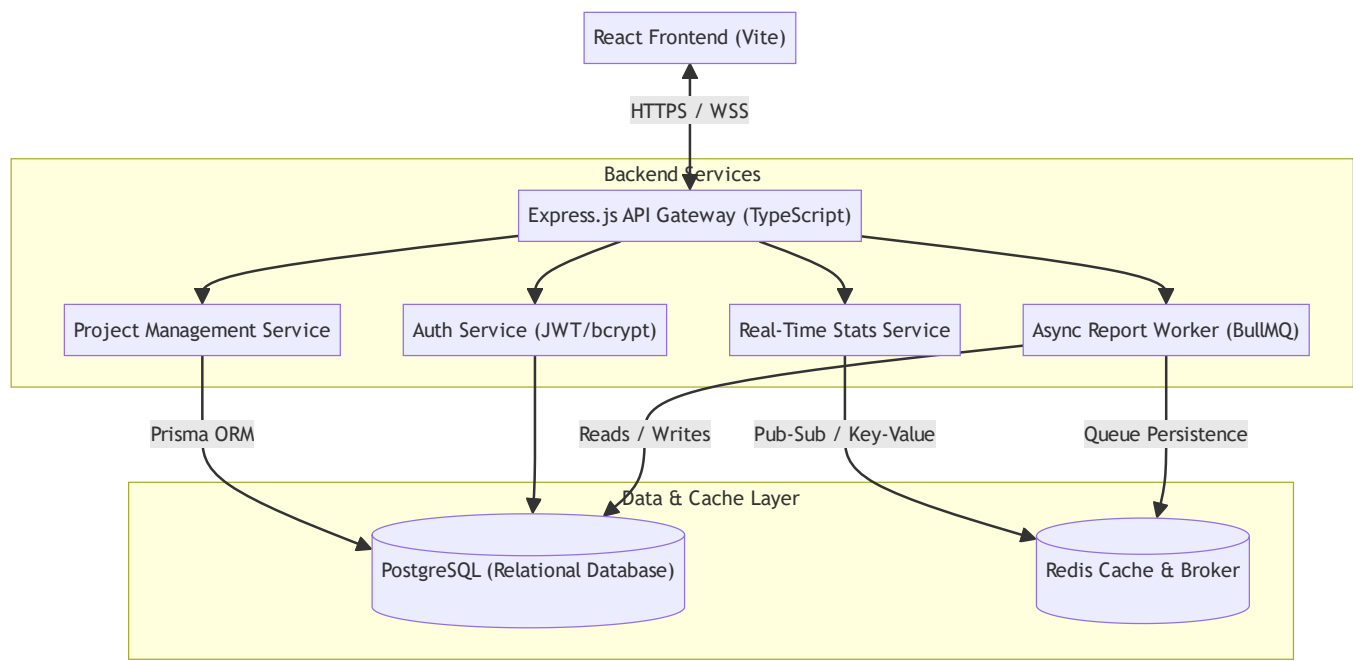


Backend Architecture & API Roadmap

This document outlines the projected backend architecture, REST/WebSocket API design, database schemas, and system dependencies needed to transition the current frontend mockups into a scalable, production-ready full-stack application.

Proposed System Architecture



Projected Backend Dependencies

Below are the recommended production-grade dependencies to build a scalable TypeScript-based backend.

Core Framework & Development

Dependency	Recommended Version	Purpose
<code>express</code>	<code>^4.19.0</code> (or <code>fastify</code>)	Lightweight, extensible router for REST endpoints.
<code>typescript</code>	<code>^5.4.0</code>	Strongly typed code compiled to vanilla JavaScript.
<code>ts-node-dev</code>	<code>^2.0.0</code>	Live reload tool for development of TypeScript servers.
<code>cors</code>	<code>^2.8.5</code>	Configures Cross-Origin Resource Sharing for the React client.

Database & ORM

Dependency	Recommended Version	Purpose
<code>prisma</code>	<code>^5.11.0</code>	Next-generation Node.js/TypeScript ORM for easy schema design & migrations.
<code>@prisma/client</code>	<code>^5.11.0</code>	Type-safe auto-generated database client.
<code>redis</code>	<code>^4.6.13</code>	High-speed caching server for live user counts, rate limits, and queues.

Real-Time & Networking

Dependency	Recommended Version	Purpose
<code>socket.io</code>	<code>^4.7.5</code> (or <code>ws</code>)	Bi-directional, real-time communication channel for live telemetry graphs.

Authentication & Security

Dependency	Recommended Version	Purpose
<code>jsonwebtoken</code>	<code>^9.0.2</code>	Generates secure access tokens for session-based auth.
<code>bcryptjs</code>	<code>^2.4.3</code>	Cryptographically hashes user passwords.
<code>helmet</code>	<code>^7.1.0</code>	Protects the Express API by setting various HTTP headers.
<code>express-rate-limit</code>	<code>^7.2.0</code>	Rate-limits repetitive requests to endpoints (e.g., authentication).

Background Tasks & Validation

Dependency	Recommended Version	Purpose
<code>bullmq</code>	<code>^5.5.0</code>	Robust Redis-backed message queue for heavy CPU jobs (like PDF generation).
<code>zod</code>	<code>^3.22.4</code>	Strict API request body validation and automatic TypeScript inference.
<code>pdfkit</code> / <code>csv-stringify</code>	<code>^0.15.0</code>	Dynamic report compilation engines.

Proposed Database Schema (Prisma Format)

The backend will require a relational database (PostgreSQL is recommended). Below is the proposed `schema.prisma` file modeling users, projects, reports, system metrics, and workspace settings.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum Role {
  USER
  MEMBER
  ADMIN
}

enum ProjectStatus {
  NOT_STARTED
  IN_PROGRESS
  IN_REVIEW
  ACTIVE_CODING
  COMPLETED
}

enum ReportStatus {
  PENDING
  PROCESSING
  COMPLETED
  FAILED
}

model User {
  id          String      @id @default(uuid())
  email       String      @unique
  passwordHash String
  name        String
  role        Role        @default(MEMBER)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  projects    Project[]   @relation("ProjectOwnership")
  reports     Report[]
  settings    WorkspaceSettings?
}

model Project {
  id          String      @id @default(uuid())
  name        String
  ownerId     String
  owner       User        @relation("ProjectOwnership", fields: [ownerId], references: [id],
  status      ProjectStatus @default(NOT_STARTED)
```

```
    progress      Int           @default(0) // Percentage 0-100
    createdAt      DateTime      @default(now())
    updatedAt      DateTime      @updatedAt
  }

model Report {
  id              String         @id @default(uuid())
  title           String
  type            String         // e.g. "Telemetry Summary", "Security Audits"
  status          ReportStatus   @default(PENDING)
  fileUrl         String?        // S3 or local storage URL for downloaded report
  fileSizeKb      Int?
  requestedBy     String
  requester       User           @relation(fields: [requestedBy], references: [id], onDelete: Cascade)
  createdAt       DateTime      @default(now())
  completedAt     DateTime?
}

model WorkspaceSettings {
  id              String         @id @default(uuid())
  userId          String         @unique
  user            User           @relation(fields: [userId], references: [id], onDelete: Cascade)
  routerTransitions Boolean      @default(true)
  routerTelemetry Boolean      @default(true)
  routerCache     Boolean      @default(false)
  updatedAt       DateTime      @updatedAt
}
```

🔑 API Endpoint Blueprint

All endpoints should follow RESTful patterns and return payloads in JSON format.

🔒 1. Authentication Services

- `POST /api/v1/auth/register` — Creates a new account.
- `POST /api/v1/auth/login` — Verifies credentials and returns a secure JWT (or sets an HttpOnly cookie).
- `POST /api/v1/auth/logout` — Invalidates the user session.
- `GET /api/v1/auth/me` — Fetches current user profile and role details.

📊 2. Dashboard & Performance metrics

- `GET /api/v1/dashboard/stats` — Aggregates total revenue, current active session count, and overall project completion percentages.
- `GET /api/v1/analytics/throughput` — Returns historical request throughput data array over the last 12 intervals (used for mounting the SVG Analytics path).

- `POST /api/v1/analytics/simulate-spike` — Triggers a simulated telemetry API traffic spike for profiling response charts.

3. Project Management

- `GET /api/v1/projects` — Fetches all deliverables. Supports `?search=query` for real-time table filtering.
- `POST /api/v1/projects` — Adds a new deliverable.
- `PATCH /api/v1/projects/:id` — Modifies project progress percentage or transition status.
- `DELETE /api/v1/projects/:id` — Deletes a project.

4. Asynchronous System Reports

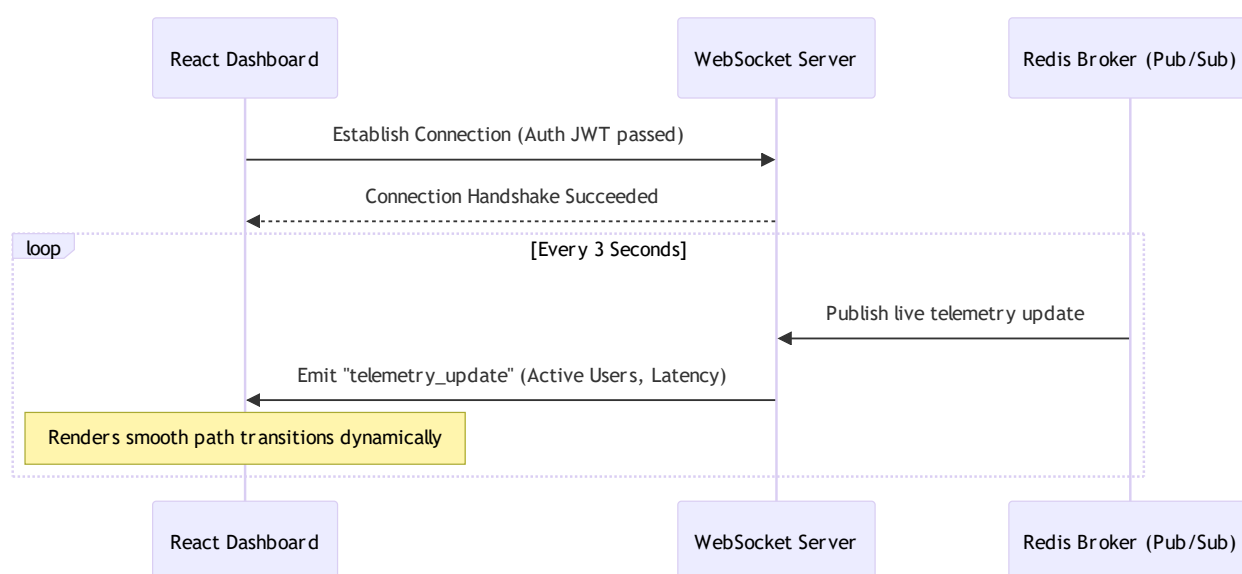
- `GET /api/v1/reports` — Lists historical downloads.
- `POST /api/v1/reports/generate` — Enqueues an offline job in BullMQ to compile reports (CSV/PDF) asynchronously. Returns task ID immediately (`202 Accepted`).
- `GET /api/v1/reports/status/:id` — Checks task status (`PENDING` → `COMPLETED`).
- `GET /api/v1/reports/download/:id` — Downloads the generated report file.

5. Workspace Preferences

- `GET /api/v1/settings` — Returns user-specific UI settings.
- `PUT /api/v1/settings` — Updates configurations like transitions, developer logs, or theme caching flags.

⚡ Real-Time System Implementation

Instead of frontend-simulated random intervals, real-time charts can be bound dynamically using standard WebSocket channels.



TIP

****Redis Pub-Sub**** is highly recommended here: it allows horizontal scaling of Node instances because any server instance can capture notifications/telemetry and broadcast it to connected clients.